EpiScope

Team Work Guide: What to Do and How to Split It Among 3 People

CSE 4345 Big Data Analytics | Complex Engineering Problem | Spring 2026

At a glance	
Duration	14 calendar days
Team size	3 students, one lead role each
Deliverables	H1 Git repo tagged v1.0, H2 IEEE report (max 15 pages + appendices), H3 video (15-20 min)
Milestones	M1 Proposal (end Day 3), M2 Checkpoint tag (end Day 7), M3 Handover (end Day 14)
Marks	70 group + 30 individual = 100 (bonus up to +5)

Most important rule

Every number in your report must come from your own runs. The instructor will run your repository on a hidden 14-day data drop using only your README, and may hold a 10-minute oral check per member. AI tools are allowed, but AI_USAGE.md is mandatory and you must be able to explain every line you submit.

1. What the assignment is asking

You are building **EpiScope**, a disease early-warning platform for all 64 districts of Bangladesh. The Ministry of Health wants to detect outbreaks early, forecast demand, and decide how to split a limited supply budget. It has four parts:

Part	What you build	Main tools
T1 Data pipeline	Generate synthetic patient records, load them into bronze / silver / gold layers, clean them, quarantine bad rows	Apache Spark, Parquet or Delta, MinIO or HDFS
T2 Quality and privacy	12+ quality rules, lineage back to source files, hashed patient IDs, small-count suppression	Spark, Python tests
T3 Analytics	Outbreak detection, 1-4 week forecasts, supply allocation optimiser, seasonal calendar, each tested against baselines	EARS / CUSUM, negative-binomial models, PuLP (LP)
T4 Dashboard	Interactive D3.js page with 5 linked views	D3.js v7, TopoJSON
T5 Handover	Git repo, IEEE report with policy brief, video	GitHub, IEEE template

Two things trip groups up. First, every member presents their own area in the video for at least 4 minutes, graded individually (15 marks), so you cannot hand off a part and forget it. Second, every member must also commit code outside their own role, and the commit history is compared against your contribution matrix.

2. The three roles

Role	Owns	Requirement IDs
Member 1 Data Engineer	Synthetic generator, ingestion, lakehouse layout, incremental loads, quality rules, lineage, de-identification, scaling experiment, Spark optimisations, orchestration	ING-*, DQ-*, PRV-1, ENG-1 to ENG-3
Member 2 Analytics Engineer	Outbreak detection, forecasting, backtests, demand conversion, optimiser, efficiency-equity curve, preparedness calendar	AN-1 to AN-10
Member 3 Visualisation Engineer	Export layer, small-cell suppression, role-based views, D3.js dashboard, what-if panel, usability test with 2 non-CS users	VIS-*, PRV-2, PRV-3

Choose roles by interest. Member 1 carries the heaviest early workload, because Members 2 and 3 cannot test on real data until the pipeline produces it. Members 2 and 3 should work on small mock data in the meantime.

3. Step-by-step plan

Phase 1: Days 1 to 3 (everyone) - M1 Proposal

Output: `docs/proposal.md`, maximum 2 pages. Repo shared with the instructor, all three members have committed.

- **All three:** agree roles, create a private GitHub repo, share it with the instructor, each make at least one commit.
- **Member 1:** write and commit the generator schema (the encounter fields from Section 5.2 of the brief). Required by Day 3.
- **Member 2:** choose diseases (dengue and diarrhoeal are the easiest pair) and methods (EARS C2 plus CUSUM for detection, negative-binomial model for forecasting).
- **Member 3:** write the stakeholder and conflict analysis (C1 to C5) and create the empty dashboard folder.
- **Everyone:** contribute 5+ risks to the risk register.

Phase 2: Days 2 to 7 (build the foundation) - M2 Checkpoint

Output: Git tag `checkpoint`. At scale factor SF = 0.25, data must flow bronze to silver to gold, with a quality report and one D3 view reading real gold output.

MEMBER 1 Data Engineer (critical path)

1. Download the real data: boundaries (geoBoundaries), population (BBS census or WorldPop), daily weather (Open-Meteo API). Cache every API response.
2. Write the generator. Seeded and deterministic. Negative-binomial counts, dengue driven by real weather with a 4-8 week lag, 30+ injected outbreaks, and a separate ground-truth file the detector never reads.
3. Inject the required defects: about 2% duplicate encounters, 0.5% orphan facility keys, 1% invalid ages, and one facility that goes silent for 3 weeks then back-fills.
4. Write bronze ingestion, adding source, load time, batch ID and file checksum.
5. Write the silver layer: 12+ quality rules, a quarantine table with reason codes, salted hash on patient_id (salt kept outside the repo).
6. Write the gold layer: epi-week by district by disease, with lagged weather columns and the bronze batch IDs carried through.

MEMBER 2 Analytics Engineer

While waiting for real data, work on a small fake CSV with the same columns as the gold table.

1. Write EARS C2 as a Spark window function (7-week baseline, 2-week guard band).
2. Write the forecasting model: one negative-binomial model per district, trained in parallel with applyInPandas, with lagged rainfall and temperature.
3. Write the backtest harness skeleton (rolling origin, seasonal-naive baseline).

MEMBER 3 Visualisation Engineer

1. Download the boundaries and simplify them with mapshaper into TopoJSON.
2. Build view 1 (the choropleth) using dummy JSON in the agreed format.
3. On Day 5-6, agree the JSON schema with Members 1 and 2 (see Section 4).

Phase 3: Days 6 to 12 (the main work)

MEMBER 1 Data Engineer

- **Incremental, idempotent loads:** re-running a day must create no duplicates, and late records (up to 21 days) must update the affected earlier weeks.
- **Lineage:** carry bronze batch IDs through silver to gold.
- **Cross-facility de-duplication:** same hashed patient, same disease, within 7 days; measure precision on the injected duplicates.
- **Optional streaming path** with Structured Streaming (alerts within 5 minutes). Do this only if the Must items are done.
- **Write your own mock hidden drop**: an unseen facility, a 30-day-late record, a duplicate batch. Run it as a test from Day 9.

MEMBER 2 Analytics Engineer

- **Detection evaluation:** score against the ground-truth file: sensitivity, false alerts per district-year, median days to detection. Plot the threshold trade-off curve.
- **Artefact guard:** show that a completeness gate stops the silent-then-back-fill facility from raising a false alert. Compare a naive and a gated detector side by side.
- **Backtest:** rolling origin over the last 52 weeks against seasonal-naive. Report MAE and 80% interval coverage (WIS recommended).
- **Demand conversion:** convert forecasts to beds, IV saline, NS1 kits. State every assumption.
- **Optimiser:** linear programme with budget, equity floor and transport constraints. Compare against population-proportional allocation.
- **Efficiency-equity curve** over at least 5 budget levels, and a recommended equity floor.
- **Seasonal calendar** from STL decomposition, per division and disease.

MEMBER 3 Visualisation Engineer

- **Export job with small-cell suppression:** no count from 1 to 4 may appear, and a hidden cell must not be recoverable by subtraction. Write a test that fails the build if any value in 1-4 appears.
- **Views 2 to 5:** time series with brush and forecast bands, seasonal heatmap, allocation bars, ranked early-warning table.
- **Linked interaction:** one shared state object so clicking a district updates every view.
- **What-if panel:** budget, threshold and equity sliders reading a precomputed scenario grid.
- **Payload check:** total browser data at most 10 MB, aggregated only, never line-level.
- **Usability test** with at least 2 non-CS people, and record what you changed afterwards.

Phase 4: Days 11 to 14 (finish)

Day	Task	Who
11-12	Scaling experiment at SF 0.25, 0.5 and 1.0. Record per-stage runtime and output sizes. SF = 1 must finish in 45 minutes or less on a 16 GB machine.	Member 1
11-12	Two or more Spark optimisations with before/after numbers and Spark UI screenshots (broadcast join, Dhaka skew, partition pruning, compaction).	Member 1
12	**Code freeze.** Verify `make all` works from a clean clone. Tag `v1.0` only after the full pipeline runs end to end.	All

Day	Task	Who
12-13	Report: Member 1 writes Section 4 and 6.4-6.6; Member 2 writes Section 5 and 6.1-6.3; Member 3 writes Sections 7 and 8; all contribute to 1-3 and 9. Move to the IEEE template on Day 12, not Day 14.	All
13	Policy brief (2 pages, no jargon), contribution matrix, individual reflections (half a page each).	All
13-14	Video: each person records their own 4+ minutes. Show a clean run, an incremental day, a re-run proving no duplicates, and a quarantine example.	All
14	Submit one LMS form: repo URL, report PDF, video link. Check the video link in a private browser window.	All

4. Shared contracts: agree these early

The three of you only fit together at two points. Write them down on Day 5-6. Any later change should be a pull request all three review. Most 14-day projects fail when the dashboard is built against a schema that changed on Day 12.

Contract	Between	Contains
1. Gold table schema	Member 1 and Member 2	epi_year, epi_week, division, district, disease_group, cases, admissions, deaths, population, incidence_per_100k, reporting_completeness, lagged weather columns, src_batch_ids. Partitioned by epi_year and division.
2. Serving JSON schema	Member 2 and Member 3	File names and fields for: district TopoJSON, weekly series, forecasts with 80% bounds, alerts, allocation scenario grid, calendar, data-quality report, and a meta file with the freshness timestamp.

5. Where the marks are

Criterion	Marks	What earns it
R1 Problem analysis	8	All five conflicts analysed with numbers; P1-P7 mapped to real design decisions; realistic risk register
R2 Data engineering	16	All Must items; idempotent loads proven by tests; hidden test drop runs from the README unchanged
R3 Analytics	16	Methods justified; beats baselines in proper backtests; artefact handled; equity analysis leads to a clear recommendation
R4 Dashboard	12	5 well-linked D3 views; uncertainty and suppression shown clearly; tested with users
R5 Performance	6	Scaling study plus 2+ optimisations with Spark UI evidence and root-cause reasoning
R6 Societal impact	5	Concrete privacy, equity and misuse analysis; honest, actionable policy brief
R7 Communication	7	Clear report; video follows all six chapters and shows the real system running
R8 Your video segment	15 (individual)	Explain the design, code and trade-offs of your own area, and link it to the rest
R9 Contribution	15 (individual)	Sustained commits across the full two weeks that match the contribution matrix

Your group mark is also multiplied by a contribution factor based on the agreed share in the contribution matrix. Equal shares give a factor of 1.0.

6. Things that cost marks

Mistake	Why it hurts	Avoid it by
Hard-coding dates, file names, row counts or district lists	The instructor runs your pipeline on a hidden 14-day drop. Hard-coded values break it.	Derive the processing window from the data. Write your own mock drop and test it.
Reporting numbers you did not measure	Results must match tagged v1.0 code; an oral check can expose this.	Fill every result table from a real run and re-run after any code change.

Mistake	Why it hurts	Avoid it by
One person committing everything on Day 13	Contribution factor and R9 both depend on commit history.	Everyone commits throughout; review the matrix on Day 7.
Detection code reading the ground-truth file	Makes evaluation meaningless.	Add a unit test that fails if it does.
Secrets, salts or data over 50 MB in the repo	Explicit rule violation.	Commit .env.example only; git-ignore data.
Video link needing sign-in, or longer than 20 minutes	Counts as a missing submission.	Check the link in a private window before submitting.
Missing AI_USAGE.md	Required; code you cannot explain earns no credit.	Log tools, purposes and representative prompts as you go.

Minimum viable path if time runs short

Finish all **M (Must)** items first. They alone earn a solid pass. Then add **S (Should)** items such as the Spark optimisation evidence, HDFS or MinIO storage, two detection methods, the efficiency-equity curve, role-based views and the what-if panel. Skip the **C (Could)** bonus items unless everything else is done.

7. Per-person checklists

Tick these off as you go. Each item is a Must requirement unless marked (S) for Should.

MEMBER 1 - Data Engineer

[ ]	ING-1 Generator at SF 0.25 / 0.5 / 1, deterministic by seed, 25M+ encounters at SF = 1
[ ]	ING-1 Bronze ingest with source, load time, batch ID, checksum
[ ]	ING-2 All heavy work in Spark (pandas only for tiny reference data)
[ ]	ING-3 Silver and gold as Parquet or Delta, justified partitioning, registered in a catalog
[ ]	ING-4 Idempotent daily loads; late records (up to 21 days) update earlier weeks
[ ]	ING-5 (S) Storage on MinIO or HDFS rather than local disk
[ ]	DQ-1 12+ quality rules with a quarantine table and reason codes
[ ]	DQ-2 Per-load data-quality report stored as a gold table
[ ]	DQ-3 Lineage from every gold row to bronze batch IDs
[ ]	DQ-4 (S) Cross-facility patient de-duplication with measured precision
[ ]	PRV-1 Salted hash on patient_id, salt outside the repo, age bands instead of birth dates
[ ]	ENG-1 One command rebuilds everything from a clean clone; pinned dependencies
[ ]	ENG-2 Scaling experiment with per-stage runtimes and hardware stated
[ ]	ENG-3 Two or more Spark optimisations with before/after and Spark UI screenshots
[ ]	Own mock "hidden drop" test written and passing

MEMBER 2 - Analytics Engineer

[ ]	AN-1 Outbreak detection (EARS C2 or CUSUM) running in Spark
[ ]	AN-2 Evaluation vs ground truth: sensitivity, false alerts per district-year, median days to detection, threshold curve
[ ]	AN-3 (S) Two methods compared; back-fill artefact not flagged
[ ]	AN-4 1-4 week forecasts for 2+ diseases with lagged weather, trained per district in parallel
[ ]	AN-5 Rolling-origin backtest over 52 weeks vs seasonal-naive: MAE and 80% interval coverage
[ ]	AN-6 Case forecasts converted to beds, saline, kits with stated assumptions and uncertainty
[ ]	AN-7 Allocation LP vs population-proportional baseline
[ ]	AN-8 (S) Efficiency-equity curve over 5+ budget levels and a recommended equity floor
[ ]	AN-9 Seasonal preparedness calendar per division and disease
[ ]	Detection code never reads the ground-truth file (unit test)
[ ]	Optimiser constraint tests written

MEMBER 3 - Visualisation Engineer

[ ]	VIS-1 Dashboard built only with D3.js v7 (no Plotly, Chart.js, Vega-Lite, Tableau, Power BI)
[ ]	VIS-2 Five views: choropleth with alerts, time series with brush and forecast bands, seasonal heatmap, allocation view, ranked early-warning table
[ ]	VIS-3 Linked interaction across all views; tooltips, legends and axis labels
[ ]	VIS-4 Browser loads aggregated gold output only, total 10 MB or less

[ ]	PRV-2 Small-cell suppression: nothing from 1 to 4, not recoverable by subtraction; automated test
[ ]	PRV-3 (S) Role-based views: national vs district user
[ ]	VIS-5 (S) What-if sliders for budget, threshold and equity floor
[ ]	VIS-6 (S) Colour-blind-safe palette, uncertainty always shown, suppressed cells marked, freshness timestamp on every view
[ ]	Usability test with 2+ non-CS users, changes recorded
[ ]	Dashboard runs with one command (python -m http.server or npm run serve)

EVERYONE

[ ]	README.md: video link at top, prerequisites, one-command run, how to run the hidden drop, traceability table
[ ]	AI_USAGE.md listing tools, purposes and representative prompts
[ ]	docs/proposal.md committed by Day 3
[ ]	Git tag checkpoint (Day 7) and v1.0 (Day 14)
[ ]	IEEE report, max 15 pages plus appendices; policy brief, contribution matrix, reflections
[ ]	Video 15-20 min, 1080p, each member 4+ minutes on camera with name on screen, link works without sign-in
[ ]	Each member has committed code outside their own role